

tomers, find key features of customers, such as salary, education level, job history, and so on, that help in prediction of loan default. The bank would then build a prediction model (such as a decision tree, which we study later in this chapter) using the chosen features. When a customer then applies for a loan, the features of that particular customer are fed into the model which makes a prediction, such as an estimated likelihood of loan default. The prediction is used to make business decisions, such as whether to give a loan to the customer.

Similarly, analysts may look at the past history of sales and use it to predict future sales, to make decisions on what and how much to manufacture or order, or how to target their advertising. For example, a car company may search for customer attributes that help predict who buys different types of cars. It may find that most of its small sports cars are bought by young women whose annual incomes are above \$50,000. The company may then target its marketing to attract more such women to buy its small sports cars and may avoid wasting money trying to attract other categories of people to buy those cars.

Machine-learning techniques are key to finding patterns in data and in making predictions from these patterns. The field of *data mining* combines knowledge-discovery techniques invented by machine-learning researchers with efficient implementation techniques that enable them to be used on extremely large databases. Section 11.4 discusses data mining.

The term **business intelligence (BI)** is used in a broadly similar sense to data analytics. The term **decision support** is also used in a related but narrower sense, with a focus on reporting and aggregation, but not including machine learning/data mining. Decision-support tasks typically use SQL queries to process large amounts of data. Decision-support queries can be contrasted with queries for *online transaction processing*, where each query typically reads only a small amount of data and may perform a few small updates.

11.2 Data Warehousing

Large organizations have a complex internal organization structure, and therefore different data may be present in different locations, or on different operational systems, or under different schemas. For instance, manufacturing-problem data and customer-complaint data may be stored on different database systems. Organizations often purchase data from external sources, such as mailing lists that are used for product promotions, or credit scores of customers that are provided by credit bureaus, to decide on creditworthiness of customers.¹

Corporate decision makers require access to information from multiple such sources. Setting up queries on individual sources is both cumbersome and inefficient.

¹Credit bureaus are companies that gather information about consumers from multiple sources and compute a creditworthiness score for each consumer.

Moreover, the sources of data may store only current data, whereas decision makers may need access to past data as well; for instance, information about how purchase patterns have changed in the past few years could be of great importance. Data warehouses provide a solution to these problems.

A **data warehouse** is a repository (or archive) of information gathered from multiple sources, stored under a unified schema, at a single site. Once gathered, the data are stored for a long time, permitting access to historical data. Thus, data warehouses provide the user a single consolidated interface to data, making decision-support queries easier to write. Moreover, by accessing information for decision support from a data warehouse, the decision maker ensures that online transaction-processing systems are not affected by the decision-support workload.

11.2.1 Components of a Data Warehouse

Figure 11.1 shows the architecture of a typical data warehouse and illustrates the gathering of data, the storage of data, and the querying and data analysis support. Among the issues to be addressed in building a warehouse are the following:

- **When and how to gather data.** In a **source-driven architecture** for gathering data, the data sources transmit new information, either continually (as transaction processing takes place), or periodically (nightly, for example). In a **destination-driven architecture**, the data warehouse periodically sends requests for new data to the sources.

Unless updates at the sources are “synchronously” replicated at the warehouse, the warehouse will never be quite up-to-date with the sources. Synchronous replication can be expensive, so many data warehouses do not use synchronous replication, and they perform queries only on data that are old enough that they have

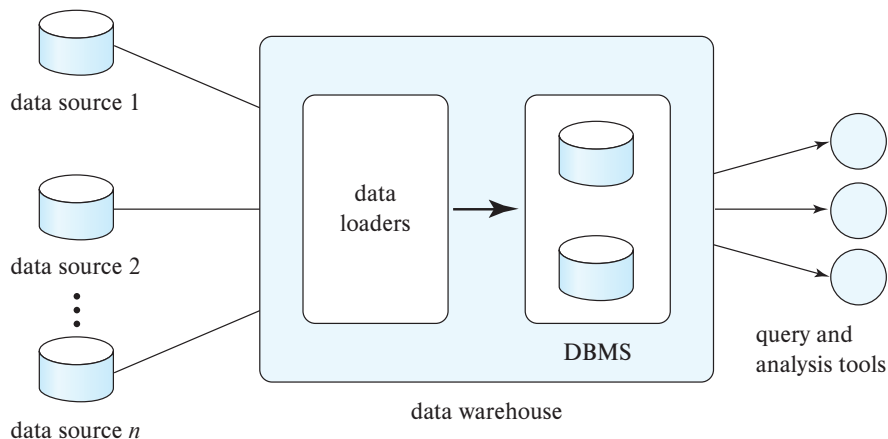


Figure 11.1 Data-warehouse architecture.

been completely replicated. Traditionally, analysts were happy with using yesterday's data, so data warehouses could be loaded with data up to the end of the previous day. However, increasingly organizations want more up-to-date data. The data freshness requirements depend on the application. Data that are within a few hours old may be sufficient for some applications; others that require real-time responses to events may use stream processing infrastructure (described in Section 10.5) instead of depending on a warehouse infrastructure.

- **What schema to use.** Data sources that have been constructed independently are likely to have different schemas. In fact, they may even use different data models. Part of the task of a warehouse is to perform schema integration and to convert data to the integrated schema before they are stored. As a result, the data stored in the warehouse are not just a copy of the data at the sources. Instead, they can be thought of as a materialized view of the data at the sources.
- **Data transformation and cleansing.** The task of correcting and preprocessing data is called **data cleansing**. Data sources often deliver data with numerous minor inconsistencies, which can be corrected. For example, names are often misspelled, and addresses may have street, area, or city names misspelled, or postal codes entered incorrectly. These can be corrected to a reasonable extent by consulting a database of street names and postal codes in each city. The approximate matching of data required for this task is referred to as **fuzzy lookup**.

Address lists collected from multiple sources may have duplicates that need to be eliminated in a **merge-purge operation** (this operation is also referred to as **deduplication**). Records for multiple individuals in a house may be grouped together so only one mailing is sent to each house; this operation is called **householding**.

Data may be **transformed** in ways other than cleansing, such as changing the units of measure, or converting the data to a different schema by joining data from multiple source relations. Data warehouses typically have graphical tools to support data transformation. Such tools allow transformation to be specified as boxes, and edges can be created between boxes to indicate the flow of data. Conditional boxes can route data to an appropriate next step in transformation.

- **How to propagate updates.** Updates on relations at the data sources must be propagated to the data warehouse. If the relations at the data warehouse are exactly the same as those at the data source, the propagation is straightforward. If they are not, the problem of propagating updates is basically the *view-maintenance* problem, which was discussed in Section 4.2.3, and is covered in more detail in Section 16.5.
- **What data to summarize.** The raw data generated by a transaction-processing system may be too large to store online. However, we can answer many queries by maintaining just summary data obtained by aggregation on a relation, rather than maintaining the entire relation. For example, instead of storing data about every sale of clothing, we can store total sales of clothing by item name and category.

The different steps involved in getting data into a data warehouse are called **extract, transform, and load** or **ETL** tasks; extraction refers to getting data from the sources, while load refers to loading the data into the data warehouse. In current generation data warehouses that support user-defined functions or MapReduce frameworks, data may be extracted, loaded into the warehouse, and then transformed. The steps are then referred to as **extract, load, and transform** or **ELT** tasks. The ELT approach permits the use of parallel processing frameworks for data transformation.

11.2.2 Multidimensional Data and Warehouse Schemas

Data warehouses typically have schemas that are designed for data analysis, using tools such as OLAP tools. The relations in a data **warehouse schema** can usually be classified as *fact tables* and *dimension tables*. **Fact tables** record information about individual events, such as sales, and are usually very large. A table recording sales information for a retail store, with one tuple for each item that is sold, is a typical example of a fact table. The attributes in fact table can be classified as either *dimension attributes* or *measure attributes*. The **measure attributes** store quantitative information, which can be aggregated upon; the measure attributes of a *sales* table would include the number of items sold and the price of the items. In contrast, **dimension attributes** are dimensions upon which measure attributes, and summaries of measure attributes, are grouped and viewed. The dimension attributes of a *sales* table would include an item identifier, the date when the item is sold, which location (store) the item was sold from, the customer who bought the item, and so on.

Data that can be modeled using dimension attributes and measure attributes are called **multidimensional data**.

To minimize storage requirements, dimension attributes are usually short identifiers that are foreign keys into other tables called **dimension tables**. For instance, a fact table *sales* would have dimension attributes *item_id*, *store_id*, *customer_id*, and *date*, and measure attributes *number* and *price*. The attribute *store_id* is a foreign key into a dimension table *store*, which has other attributes such as store location (city, state, country). The *item_id* attribute of the *sales* table would be a foreign key into a dimension table *item_info*, which would contain information such as the name of the item, the category to which the item belongs, and other item details such as color and size. The *customer_id* attribute would be a foreign key into a *customer* table containing attributes such as name and address of the customer. We can also view the *date* attribute as a foreign key into a *date_info* table giving the month, quarter, and year of each date.

The resultant schema appears in Figure 11.2. Such a schema, with a fact table, multiple dimension tables, and foreign keys from the fact table to the dimension tables is called a **star schema**. More complex data-warehouse designs may have multiple levels of dimension tables; for instance, the *item_info* table may have an attribute *manufacturer_id* that is a foreign key into another table giving details of the manufacturer. Such schemas are called **snowflake schemas**. Complex data-warehouse designs may also have more than one fact table.

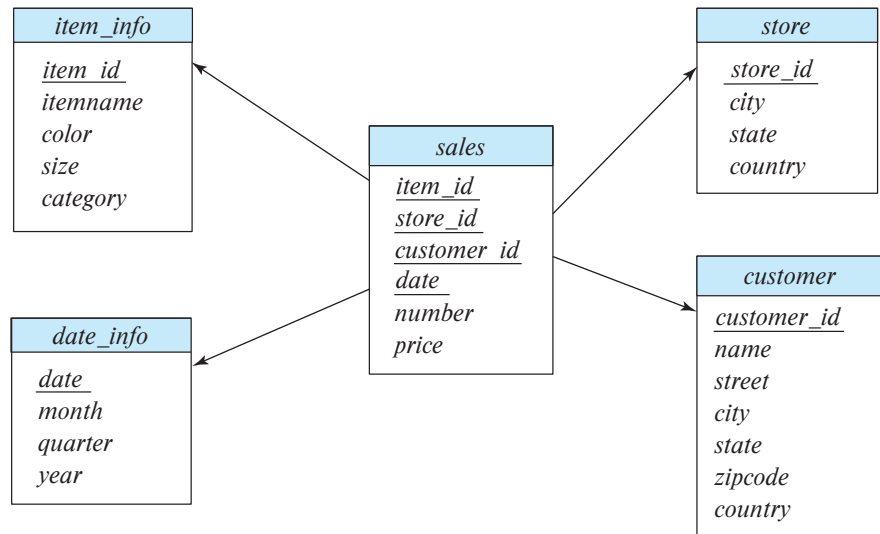


Figure 11.2 Star schema for a data warehouse.

11.2.3 Database Support for Data Warehouses

The requirements of a database system designed for transaction processing are somewhat different from one designed to support a data-warehouse system. One key difference is that a transaction-processing database needs to support many small queries, which may involve updates in addition to reads. In contrast, data warehouses typically need to process far fewer queries, but each query accesses a much larger amount of data.

Most importantly, while new records are inserted into relations in a data warehouse, and old records may be deleted once they are no longer needed, to make space for new data, records are typically never updated once they are added to a relation. Thus, data warehouses do not need to pay any overhead for concurrency control. (As described in Chapter 17 and Chapter 18, if concurrent transactions read and write the same data, the resultant data may become inconsistent. Concurrency control restricts concurrent accesses in a way that ensures there is no erroneous update to the database.) The overhead of concurrency control can be significant in terms of not just time taken for query processing, but also in terms of storage, since databases often store multiple versions of data to avoid conflicts between small update transactions and long read-only transactions. None of these overheads are needed in a data warehouse.

Databases traditionally store all attributes of a tuple together, and tuples are stored sequentially in a file. Such a storage layout is referred to as *row-oriented storage*. In contrast, in **column-oriented storage**, each attribute of a relation is stored in a separate file, with values from successive tuples stored at successive positions in the file. Assuming fixed-size data types, the value of attribute *A* of the *i*th tuple of a relation can be found

by accessing the file corresponding to attribute A and reading the value at offset $(i - 1)$ times the size (in bytes) of values in attribute A .

Column-oriented storage has at least two major benefits over row-oriented storage:

1. When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory. In contrast, in row-oriented storage, not only are irrelevant attributes fetched into memory, but they may also get prefetched into processor cache, wasting cache space and memory bandwidth, if they are stored adjacent to attributes used in the query.
2. Storing values of the same type together increases the effectiveness of compression; compression can greatly reduce both the disk storage cost and the time to retrieve data from disk.

On the other hand, column-oriented storage has the drawback that storing or fetching a single tuple requires multiple I/O operations.

As a result of these trade-offs, column-oriented storage is not widely used for transaction-processing applications. However, column-oriented storage is today widely used for data-warehousing applications, where accesses are rarely to individual tuples but rather require scanning and aggregating multiple tuples. Column-oriented storage is described in more detail in Section 13.6.

Database implementations that are designed purely for data warehouse applications include Teradata, Sybase IQ, and Amazon Redshift. Many traditional databases support efficient execution of data warehousing applications by adding features such as columnar storage; these include Oracle, SAP HANA, Microsoft SQL Server, and IBM DB2.

In the 2010s there has been an explosive growth in Big Data systems that are designed to process queries over data stored in files. Such systems are now a key part of the data warehouse infrastructure. As we saw in Section 10.3, the motivation for such systems was the growth of data generated by online systems in the form of log files, which have a lot of valuable information that can be exploited for decision support. However, these systems can handle any kind of data, including relational data. Apache Hadoop is one such system, and the Hive system allows SQL queries to be executed on top of the Hadoop system.

A number of companies provide software to optimize Hive query processing, including Cloudera and Hortonworks. Apache Spark is another popular Big Data system that supports SQL queries on data stored in files. Compressed file structures containing records with columns, such as Orc and Parquet, are increasingly used to store such log records, simplifying integration with SQL. Such file formats are discussed in more detail in Section 13.6.

11.2.4 Data Lakes

While data warehouses pay a lot of attention to ensuring a common data schema to ease the job of querying the data, there are situations where organizations want to store data without paying the cost of creating a common schema and transforming data to the common schema. The term **data lake** is used to refer to a repository where data can be stored in multiple formats, including structured records and unstructured file formats. Unlike data warehouses, data lakes do not require up-front effort to preprocess data, but they do require more effort when creating queries. Since data may be stored in many different formats, querying tools also need to be quite flexible. Apache Hadoop and Apache Spark are popular tools for querying such data, since they support querying of both unstructured and structured data.

11.3 Online Analytical Processing

Data analysis often involves looking for patterns that arise when data values are grouped in “interesting” ways. As a simple example, summing credit hours for each department is a way to discover which departments have high teaching responsibilities. In a retail business, we might group sales by product, the date or month of the sale, the color or size of the product, or the profile (such as age group and gender) of the customer who bought the product.

11.3.1 Aggregation on Multidimensional Data

Consider an application where a shop wants to find out what kinds of clothes are popular. Let us suppose that clothes are characterized by their *item_name*, *color*, and *size*, and that we have a relation *sales* with the schema.

sales (*item_name*, *color*, *clothes_size*, *quantity*)

Suppose that *item_name* can take on the values (skirt, dress, shirt, pants), *color* can take on the values (dark, pastel, white), *clothes_size* can take on values (small, medium, large), and *quantity* is an integer value representing the total number of items of a given {*item_name*, *color*, *clothes_size*}. An instance of the *sales* relation is shown in Figure 11.3.

Statistical analysis often requires grouping on multiple attributes. The attribute *quantity* of the *sales* relation is a measure attribute, since it measures the number of units sold, while *item_name*, *color*, and *clothes_size* are dimension attributes. (A more realistic version of the *sales* relation would have additional dimensions, such as time and sales location, and additional measures such as monetary value of the sale.)

To analyze the multidimensional data, a manager may want to see data laid out as shown in the table in Figure 11.4. The table shows total quantities for different combinations of *item_name* and *color*. The value of *clothes_size* is specified to be **all**, indicating that the displayed values are a summary across all values of *clothes_size* (i.e., we want to group the “small,” “medium,” and “large” items into one single group.

<i>item_name</i>	<i>color</i>	<i>clothes_size</i>	<i>quantity</i>
dress	dark	small	2
dress	dark	medium	6
dress	dark	large	12
dress	pastel	small	4
dress	pastel	medium	3
dress	pastel	large	3
dress	white	small	2
dress	white	medium	3
dress	white	large	0
pants	dark	small	14
pants	dark	medium	6
pants	dark	large	0
pants	pastel	small	1
pants	pastel	medium	0
pants	pastel	large	1
pants	white	small	3
pants	white	medium	0
pants	white	large	2
shirt	dark	small	2
shirt	dark	medium	6
shirt	dark	large	6
shirt	pastel	small	4
shirt	pastel	medium	1
shirt	pastel	large	2
shirt	white	small	17
shirt	white	medium	1
shirt	white	large	10
skirt	dark	small	2
skirt	dark	medium	5
skirt	dark	large	1
skirt	pastel	small	11
skirt	pastel	medium	9
skirt	pastel	large	15
skirt	white	small	2
skirt	white	medium	5
skirt	white	large	3

Figure 11.3 An example of *sales* relation.

The table in Figure 11.4 is an example of a **cross-tabulation** (or **cross-tab**, for short), also referred to as a **pivot-table**. In general, a cross-tab is a table derived from a relation

<i>clothes_size</i> all		<i>color</i>			
<i>item_name</i>		dark	pastel	white	total
	skirt	8	35	10	53
	dress	20	10	5	35
	shirt	14	7	28	49
	pants	20	2	5	27
	total	62	54	48	164

Figure 11.4 Cross-tabulation of *sales* by *item_name* and *color*.

(say R), where values for one attribute (say A) form the column headers and values for another attribute (say B) form the row header. For example, in Figure 11.4, the attribute *color* corresponds to A (with values “dark,” “pastel,” and “white”), and the attribute *item_name* corresponds to B (with values “skirt,” “dress,” “shirt,” and “pants”).

Each cell in the pivot-table can be identified by (a_i, b_j) , where a_i is a value for A and b_j a value for B . The values of the various cells in the pivot-table are derived from the relation R as follows: If there is at most one tuple in R with any (a_i, b_j) value, the value in the cell is derived from that single tuple (if any); for instance, it could be the value of one or more other attributes of the tuple. If there can be multiple tuples with an (a_i, b_j) value, the value in the cell must be derived by aggregation on the tuples with that value. In our example, the aggregation used is the sum of the values for attribute *quantity*, across all values for *clothes_size*, as indicated by “*clothes_size*: **all**” above the cross-tab in Figure 11.4. Thus, the value for cell (skirt, pastel) is 35, since there are three tuples in the *sales* table that meet that criteria, with values 11, 9, and 15.

In our example, the cross-tab also has an extra column and an extra row storing the totals of the cells in the row/column. Most cross-tabs have such summary rows and columns.

The generalization of a cross-tab, which is two-dimensional, to n dimensions can be visualized as an n -dimensional cube, called the **data cube**. Figure 11.5 shows a data cube on the *sales* relation. The data cube has three dimensions, *item_name*, *color*, and *clothes_size*, and the measure attribute is *quantity*. Each cell is identified by values for these three dimensions. Each cell in the data cube contains a value, just as in a cross-tab. In Figure 11.5, the value contained in a cell is shown on one of the faces of the cell; other faces of the cell are shown blank if they are visible. All cells contain values, even if they are not visible. The value for a dimension may be **all**, in which case the cell contains a summary over all values of that dimension, as in the case of cross-tabs.

The number of different ways in which the tuples can be grouped for aggregation can be large. In the example of Figure 11.5, there are 3 colors, 4 items, and 3 sizes resulting in a cube size of $3 \times 4 \times 3 = 36$. Including the summary values, we obtain a

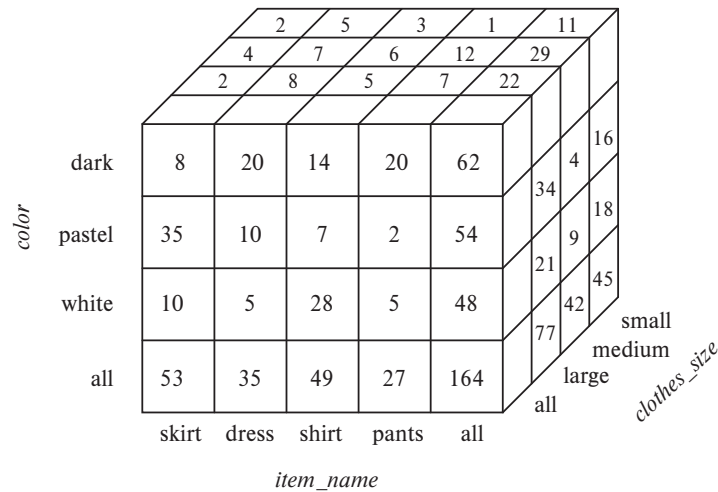


Figure 11.5 Three-dimensional data cube.

$4 \times 5 \times 4$ cube, whose size is 80. In fact, for a table with n dimensions, aggregation can be performed with grouping on each of the 2^n subsets of the n dimensions.²

An **online analytic processing** (OLAP) system allows a data analyst to look at different cross-tabs on the same data by *interactively* selecting the attributes in the cross-tab. Each cross-tab is a two-dimensional view on a multidimensional data cube. For instance, the analyst may select a cross-tab on *item_name* and *clothes_size* or a cross-tab on *color* and *clothes_size*. The operation of changing the dimensions used in a cross-tab is called **pivoting**.

OLAP systems allow an analyst to see a cross-tab on *item_name* and *color* for a fixed value of *clothes_size*, for example, large, instead of the sum across all sizes. Such an operation is referred to as **slicing**, since it can be thought of as viewing a slice of the data cube. The operation is sometimes called **dicing**, particularly when values for multiple dimensions are fixed.

When a cross-tab is used to view a multidimensional cube, the values of dimension attributes that are not part of the cross-tab are shown above the cross-tab. The value of such an attribute can be **all**, as shown in Figure 11.4, indicating that data in the cross-tab are a summary over all values for the attribute. Slicing/dicing simply consists of selecting specific values for these attributes, which are then displayed on top of the cross-tab.

OLAP systems permit users to view data at any desired level of granularity. The operation of moving from finer-granularity data to a coarser granularity (by means of aggregation) is called a **rollup**. In our example, starting from the data cube on the

²Grouping on the set of all n dimensions is useful only if the table may have duplicates.

sales table, we got our example cross-tab by rolling up on the attribute *clothes_size*. The opposite operation—that of moving from coarser-granularity data to finer-granularity data—is called a **drill down**. Finer-granularity data cannot be generated from coarse-granularity data; they must be generated either from the original data or from even finer-granularity summary data.

Analysts may wish to view a dimension at different levels of detail. For instance, consider an attribute of type **datetime** that contains a date and a time of day. Using time precise to a second (or less) may not be meaningful: An analyst who is interested in rough time of day may look at only the hour value. An analyst who is interested in sales by day of the week may map the date to a day of the week and look only at that. Another analyst may be interested in aggregates over a month, or a quarter, or for an entire year.

The different levels of detail for an attribute can be organized into a **hierarchy**. Figure 11.6a shows a hierarchy on the **datetime** attribute. As another example, Figure 11.6b shows a hierarchy on location, with the city being at the bottom of the hierarchy, state above it, country at the next level, and region being the top level. In our earlier example, clothes can be grouped by category (for instance, menswear or womenswear); *category* would then lie above *item_name* in our hierarchy on clothes. At the level of actual values, skirts and dresses would fall under the womenswear category and pants and shirts under the menswear category.

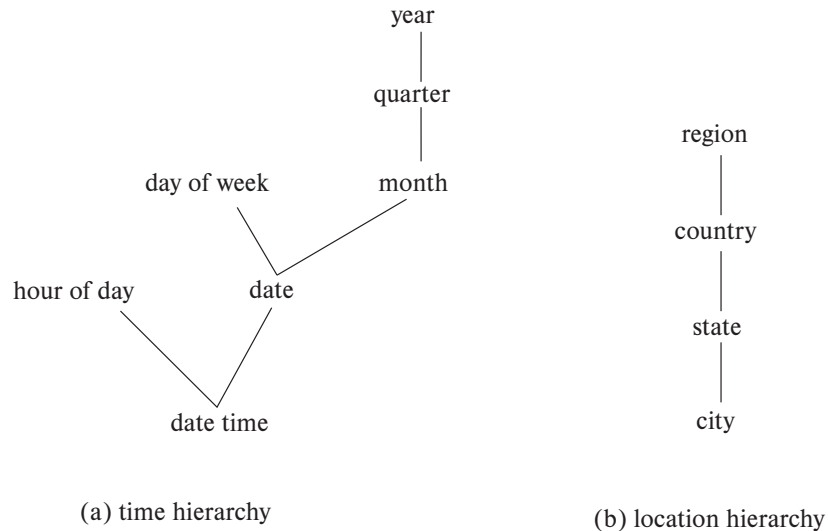


Figure 11.6 Hierarchies on dimensions.

clothes_size: **all**

		<i>color</i>				
<i>category</i>	<i>item_name</i>	dark	pastel	white	total	
womenswear	skirt	8	8	10	53	
	dress	20	20	5	35	
	subtotal	28	28	15		88
menswear	pants	14	14	28	49	
	shirt	20	20	5	27	
	subtotal	34	34	33		76
total		62	62	48		164

Figure 11.7 Cross-tabulation of *sales* with hierarchy on *item_name*.

An analyst may be interested in viewing sales of clothes divided as menswear and womenswear, and not interested in individual values. After viewing the aggregates at the level of womenswear and menswear, an analyst may *drill down the hierarchy* to look at individual values. An analyst looking at the detailed level may *drill up the hierarchy* and look at coarser-level aggregates. Both levels can be displayed on the same cross-tab, as in Figure 11.7.

11.3.2 Relational Representation of Cross-Tabs

A cross-tab is different from relational tables usually stored in databases, since the number of columns in the cross-tab depends on the actual data. A change in the data values may result in adding more columns, which is not desirable for data storage. However, a cross-tab view is desirable for display to users. It is straightforward to represent a cross-tab without summary values in a relational form with a fixed number of columns. A cross-tab with summary rows/columns can be represented by introducing a special value **all** to represent subtotals, as in Figure 11.8. The SQL standard actually uses the *null* value in place of **all**, but to avoid confusion with regular null values, we shall continue to use **all**.

Consider the tuples (skirt, **all**, **all**, 53) and (dress, **all**, **all**, 35). We have obtained these tuples by eliminating individual tuples with different values for *color* and *clothes_size*, and by replacing the value of *quantity* with an aggregate—namely, the sum of the quantities. The value **all** can be thought of as representing the set of all values for an attribute. Tuples with the value **all** for the *color* and *clothes_size* dimensions can be obtained by an aggregation on the *sales* relation with a **group by** on the column *item_name*. Similarly, a **group by** on *color*, *clothes_size* can be used to get the tuples with the value **all** for *item_name*, and a **group by** with no attributes (which can simply be omitted in SQL) can be used to get the tuple with value **all** for *item_name*, *color*, and *clothes_size*.

<i>item_name</i>	<i>color</i>	<i>clothes_size</i>	<i>quantity</i>
skirt	dark	all	8
skirt	pastel	all	35
skirt	white	all	10
skirt	all	all	53
dress	dark	all	20
dress	pastel	all	10
dress	white	all	5
dress	all	all	35
shirt	dark	all	14
shirt	pastel	all	7
shirt	white	all	28
shirt	all	all	49
pants	dark	all	20
pants	pastel	all	2
pants	white	all	5
pants	all	all	27
all	dark	all	62
all	pastel	all	54
all	white	all	48
all	all	all	164

Figure 11.8 Relational representation of the data in Figure 11.4.

Hierarchies can also be represented by relations. For example, the fact that skirts and dresses fall under the womenswear category and the pants and shirts under the menswear category can be represented by a relation *itemcategory* (*item_name*, *category*). This relation can be joined with the *sales* relation to get a relation that includes the category for each item. Aggregation on this joined relation allows us to get a cross-tab with hierarchy. As another example, a hierarchy on city can be represented by a single relation *city_hierarchy* (*ID*, *city*, *state*, *country*, *region*), or by multiple relations, each mapping values in one level of the hierarchy to values at the next level. We assume here that cities have unique identifiers, stored in the attribute *ID*, to avoid confusing between two cities with the same name, for example, the Springfield in Missouri and the Springfield in Illinois.

11.3.3 OLAP in SQL

Analysts using OLAP systems need answers to multiple aggregates to be generated interactively, without having to wait for multiple minutes or hours. This led initially to the development of specialized systems for OLAP (see Note 11.1 on page 535). Many database systems now implement OLAP along with SQL constructs to express OLAP queries. As we saw in Section 5.5.3, several SQL implementations, such as Microsoft

SQL Server and Oracle, support a **pivot** clause that allows creation of cross-tabs. Given the *sales* relation from Figure 11.3, the query:

```
select *
from sales
pivot (
    sum(quantity)
    for color in ('dark','pastel','white')
)
order by item_name;
```

returns the cross-tab shown in Figure 11.9. Note that the **for** clause within the **pivot** clause specifies the *color* values that appear as attribute names in the pivot result. The attribute *color* itself does not appear in the result, although all other attributes are retained, except that the values for the newly created attributes are specified to come from the attribute *quantity*. In case more than one tuple contributes values to a given cell, the aggregate operation within the **pivot** clause specifies how the values should be combined. In the above example, the *quantity* values are summed up.

Note that the pivot clause by itself does not compute the subtotals we saw in the pivot table from Figure 11.4. However, we can first generate the relational representation shown in Figure 11.8, using a cube operation, as outlined shortly, and then apply the pivot clause on that representation to get an equivalent result. In this case, the value **all** must also be listed in the **for** clause, and the **order by** clause needs to be modified to order **all** at the end.

The data in a data cube cannot be generated by a single SQL query if we use only the basic **group by** constructs, since aggregates are computed for several different groupings

<i>item_name</i>	<i>clothes_size</i>	<i>dark</i>	<i>pastel</i>	<i>white</i>
dress	small	2	4	2
dress	medium	6	3	3
dress	large	12	3	0
pants	small	14	1	3
pants	medium	6	0	0
pants	large	0	1	2
shirt	small	2	4	17
shirt	medium	6	1	1
shirt	large	6	2	10
skirt	small	2	11	2
skirt	medium	5	9	5
skirt	large	1	15	3

Figure 11.9 Result of SQL pivot operation on the *sales* relation of Figure 11.3.

Note 11.1 OLAP IMPLEMENTATION

The earliest OLAP systems used multidimensional arrays in memory to store data cubes and are referred to as **multidimensional OLAP (MOLAP)** systems. Later, OLAP facilities were integrated into relational systems, with data stored in a relational database. Such systems are referred to as **relational OLAP (ROLAP)** systems. Hybrid systems, which store some summaries in memory and store the base data and other summaries in a relational database, are called **hybrid OLAP (HOLAP)** systems.

Many OLAP systems are implemented as client-server systems. The server contains the relational database as well as any MOLAP data cubes. Client systems obtain views of the data by communicating with the server.

A naïve way of computing the entire data cube (all groupings) on a relation is to use any standard algorithm for computing aggregate operations, one grouping at a time. The naïve algorithm would require a large number of scans of the relation. A simple optimization is to compute an aggregation on, say, *(item_name, color)* from an aggregation *(item_name, color, clothes_size)*, instead of from the original relation. The amount of data read drops significantly by computing an aggregate from another aggregate, instead of from the original relation. Further improvements are possible; for instance, multiple groupings can be computed on a single scan of the data.

Early OLAP implementations precomputed and stored entire data cubes, that is, groupings on all subsets of the dimension attributes. Precomputation allows OLAP queries to be answered within a few seconds, even on datasets that may contain millions of tuples adding up to gigabytes of data. However, there are 2^n groupings with n dimension attributes; hierarchies on attributes increase the number further. As a result, the entire data cube is often larger than the original relation that formed the data cube and in many cases it is not feasible to store the entire data cube.

Instead of precomputing and storing all possible groupings, it makes sense to precompute and store some of the groupings, and to compute others on demand. Instead of computing queries from the original relation, which may take a very long time, we can compute them from other precomputed queries. For instance, suppose that a query requires grouping by *(item_name, color)*, and this has not been precomputed. The query result can be computed from summaries by *(item_name, color, clothes_size)*, if that has been precomputed. See the bibliographical notes for references on how to select a good set of groupings for precomputation, given limits on the storage available for precomputed results.

of the dimension attributes. Using only the basic **group by** construct, we would have to write many separate SQL queries and combine them using a union operation. SQL supports special syntax to allow multiple group by operations to be specified concisely.

As we saw in Section 5.5.4, SQL supports generalizations of the **group by** construct to perform the **cube** and **rollup** operations. The **cube** and **rollup** constructs in the **group by** clause allow multiple **group by** queries to be run in a single query with the result returned as a single relation in a style similar to that of the relation of Figure 11.8.

Consider again our retail shop example and the relation:

sales (*item_name*, *color*, *clothes_size*, *quantity*)

If we want to generate the entire data cube using individual group by queries, we have to write a separate query for each of the following eight sets of group by attributes:

{ (*item_name*, *color*, *clothes_size*), (*item_name*, *color*), (*item_name*, *clothes_size*),
(*color*, *clothes_size*), (*item_name*), (*color*), (*clothes_size*), () }

where () denotes an empty **group by** list.

As we saw in Section 5.5.4, the **cube** construct allows us to accomplish this in one query:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by cube(item_name, color, clothes_size);
```

The preceding query produces a relation whose schema is:

(*item_name*, *color*, *clothes_size*, sum(*quantity*))

So that the result of this query is indeed a relation, tuples in the result contain *null* as the value of those attributes not present in a particular grouping. For example, tuples produced by grouping on *clothes_size* have a schema (*clothes_size*, sum(*quantity*)). They are converted to tuples on (*item_name*, *color*, *clothes_size*, sum(*quantity*)) by inserting *null* for *item_name* and *color*.

Data cube relations are often very large. The cube query above, with 3 possible colors, 4 possible item names, and 3 sizes, has 80 tuples. The relation of Figure 11.8 is generated by doing a **group by cube** on *item_name* and *color*, with an extra column specified in the **select** clause showing **all** for *clothes_size*.

To generate that relation in SQL, we substitute **all** for *null* using the **grouping** function, as we saw earlier in Section 5.5.4. The **grouping** function distinguishes those nulls generated by OLAP operations from “normal” nulls actually stored in the database or arising from an outer join. Recall that the **grouping** function returns 1 if its argument

is a null value generated by a **cube** or **rollup** and 0 otherwise. We may then operate on the result of a call to **grouping** using **case** expressions to replace OLAP-generated nulls with **all**. Then the relation in Figure 11.8, with occurrences of *null* replaced by **all**, can be computed by the query:

```
select case when grouping(item_name) = 1 then 'all'
         else item_name end as item_name,
       case when grouping(color) = 1 then 'all'
         else color end as color,
       'all' as clothes_size, sum(quantity) as quantity
from sales
group by cube(item_name, color);
```

The **rollup** construct is the same as the **cube** construct except that **rollup** generates fewer **group by** queries. We saw that **group by cube** (*item_name, color, clothes_size*) generated all eight ways of forming a **group by** query using some (or all or none) of the attributes. In:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by rollup(item_name, color, clothes_size);
```

the clause **group by rollup**(*item_name, color, clothes_size*) generates only four groupings:

$$\{ (item_name, color, clothes_size), (item_name, color), (item_name), () \}$$

Notice that the order of the attributes in the **rollup** makes a difference; the last attribute (*clothes_size*, in our example) appears in only one grouping, the penultimate (second last) attribute in two groupings, and so on, with the first attribute appearing in all groups but one (the empty grouping).

Why might we want the specific groupings that are used in **rollup**? These groups are of frequent practical interest for hierarchies (as in Figure 11.6, for example). For the location hierarchy (*Region, Country, State, City*), we may want to group by *Region* to get sales by region. Then we may want to “drill down” to the level of countries within each region, which means we would group by *Region, Country*. Drilling down further, we may wish to group by *Region, Country, State* and then by *Region, Country, State, City*. The **rollup** construct allows us to specify this sequence of drilling down for further detail.

As we saw earlier in Section 5.5.4, multiple **rollups** and **cubes** can be used in a single **group by** clause. For instance, the following query:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by rollup(item_name), rollup(color, clothes_size);
```

generates the groupings:

$$\{ (item_name, color, clothes_size), (item_name, color), (item_name), \\ (color, clothes_size), (color), () \}$$

To understand why, observe that **rollup**(*item_name*) generates two groupings, {(*item_name*), ()}, and **rollup**(*color*, *clothes_size*) generates three groupings, {(*color*, *clothes_size*), (*color*), ()}. The Cartesian product of the two gives us the six groupings shown.

Neither the **rollup** nor the **cube** clause gives complete control on the groupings that are generated. For instance, we cannot use them to specify that we want only groupings {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}. Such restricted groupings can be generated by using the **grouping sets** construct, in which one can specify the specific list of groupings to be used. To obtain only groupings {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}, we would write:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by grouping sets ((color, clothes_size), (clothes_size, item_name));
```

Specialized languages have been developed for querying multidimensional OLAP schemas, which allow some common tasks to be expressed more easily than with SQL. These include the MDX and DAX query languages developed by Microsoft.

11.3.4 Reporting and Visualization Tools

Report generators are tools to generate human-readable summary reports from a database. They integrate querying the database with the creation of formatted text and summary charts (such as bar or pie charts). For example, a report may show the total sales in each of the past 2 months for each sales region.

The application developer can specify report formats by using the formatting facilities of the report generator. Variables can be used to store parameters such as the month and the year and to define fields in the report. Tables, graphs, bar charts, or other graphics can be defined via queries on the database. The query definitions can make use of the parameter values stored in the variables.

Once we have defined a report structure on a report-generator facility, we can store it and can execute it at any time to generate a report. Report-generator systems provide a variety of facilities for structuring tabular output, such as defining table and column headers, displaying subtotals for each group in a table, automatically splitting long tables into multiple pages, and displaying subtotals at the end of each page.

Figure 11.10 is an example of a formatted report. The data in the report are generated by aggregation on information about orders.

Report-generation tools are available from a variety of vendors, such as SAP Crystal Reports and Microsoft (SQL Server Reporting Services). Several application suites, such as Microsoft Office, provide a way of embedding formatted query results from

Acme Supply Company, Inc.
Quarterly Sales Report

Period: Jan. 1 to March 31, 2009

Region	Category	Sales	Subtotal
North	Computer Hardware	1,000,000	1,500,000
	Computer Software	500,000	
	All categories		
South	Computer Hardware	200,000	600,000
	Computer Software	400,000	
	All categories		
Total Sales			2,100,000

Figure 11.10 A formatted report.

a database directly into a document. Chart-generation facilities provided by Crystal Reports or by spreadsheets such as Excel can be used to access data from databases and to generate tabular depictions of data or graphical depictions using charts or graphs. Such charts can be embedded within text documents. The charts are created initially from data generated by executing queries against the database; the queries can be re-executed and the charts regenerated when required, to generate a current version of the overall report.

Techniques for **data visualization**, that is, graphical representation of data, that go beyond the basic chart types, are very important for data analysis. Data-visualization systems help users to examine large volumes of data and to detect patterns visually. Visual displays of data—such as maps, charts, and other graphical representations—allow data to be presented compactly to users. A single graphical screen can encode as much information as a far larger number of text screens.

For example, if the user wants to find out whether the occurrence of a disease is correlated to the locations of the patients, the locations of patients can be encoded in a special color—say, red—on a map. The user can then quickly discover locations where problems are occurring. The user may then form hypotheses about why problems are occurring in those locations and may verify the hypotheses quantitatively against the database.

As another example, information about values can be encoded as a color and can be displayed with as little as one pixel of screen area. To detect associations between pairs of items, we can use a two-dimensional pixel matrix, with each row and each column representing an item. The percentage of transactions that buy both items can be encoded by the color intensity of the pixel. Items with high association will show up as bright pixels in the screen—easy to detect against the darker background.

13.6 Column-Oriented Storage

Databases traditionally store all attributes of a tuple together in a record, and tuples are stored in a file as we have just seen. Such a storage layout is referred to as a *row-oriented storage*.

In contrast, in **column-oriented storage**, also called a **columnar storage**, each attribute of a relation is stored separately, with values of the attribute from successive tuples stored at successive positions in the file. Figure 13.14 shows how the *instructor* relation would be stored in column-oriented storage, with each attribute stored separately.

In the simplest form of column-oriented storage, each attribute is stored in a separate file. Further, each file is *compressed*, to reduce its size. (We discuss more complex schemes that store columns consecutively in a single file later in this section.)

If a query needs to access the entire contents of the i th row of a table, the values at the i th position in each of the columns are retrieved and used to reconstruct the row. Column-oriented storage thus has the drawback that fetching multiple attributes of a single tuple requires multiple I/O operations. Thus, it is not suitable for queries that fetch multiple attributes from a few rows of a relation.

However, column-oriented storage is well suited for data analysis queries, which process many rows of a relation, but often only access some of the attributes. The reasons are as follows:

- **Reduced I/O.** When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory. In contrast, in row-oriented storage, irrelevant attributes are fetched into memory from disk. The reduction in I/O can lead to significant reduction in query execution cost.

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 13.14 Columnar representation of the *instructor* relation.

- **Improved CPU cache performance.** When the query processor fetches the contents of a particular attribute, with modern CPU architectures multiple consecutive bytes, called a cache line, are fetched from memory to CPU cache. If these bytes are accessed later, access is much faster if they are in cache than if they have to be fetched from main memory. However, if these adjacent bytes contain values for attributes that are not needed by the query, fetching them into cache wastes memory bandwidth and uses up cache space that could have been used for other data. Column-oriented storage does not suffer from this problem, since adjacent bytes are from the same column, and data analysis queries usually access all these values consecutively.
- **Improved compression.** Storing values of the same type together significantly increases the effectiveness of compression, when compared to compressing data stored in row format; in the latter case, adjacent attributes are of different types, reducing the efficiency of compression. Compression significantly reduces the time taken to retrieve data from disk, which is often the highest-cost component for many queries. If the compressed files are stored in memory, the in-memory storage space is also reduced correspondingly, which is particularly important since main memory is significantly more expensive than disk storage.
- **Vector processing.** Many modern CPU architectures support **vector processing**, which allows a CPU operation to be applied in parallel on a number of elements of an array. Storing data columnwise allows vector processing of operations such as comparing an attribute with a constant, which is important for applying selection conditions on a relation. Vector processing can also be used to compute an aggregate of multiple values in parallel, instead of aggregating the values one at a time.

As a result of these benefits, column-oriented storage is increasingly used in data-warehousing applications, where queries are primarily data analysis queries. It should be noted that indexing and query processing techniques need to be carefully designed to get the performance benefits of column-oriented storage. We outline indexing and query processing techniques based on bitmap representations, which are well suited to column-oriented storage, in Section 14.9; further details are provided in Section 24.3.

Databases that use column-oriented storage are referred to as **column stores**, while databases that use row-oriented storage are referred to as **row stores**.

It should be noted that column-oriented storage does have several drawbacks, which make them unsuitable for transaction processing.

- **Cost of tuple reconstruction.** As we saw earlier, reconstructing a tuple from the individual columns can be expensive, negating the benefits of columnar representation if many columns need to be reconstructed. While tuple reconstruction is common in transaction-processing applications, data analysis applications usually

output only a few columns out of many that are stored in “fact tables” in data warehouses.

- **Cost of tuple deletion and update.** Deleting or updating a single tuple in a compressed representation would require rewriting the entire sequence of tuples that are compressed as one unit. Since updates and deletes are common in transaction-processing applications, column-oriented storage would result in a high cost for these operations if a large number of tuples were compressed as one unit.

In contrast, data-warehousing systems typically do not support updates to tuples, and instead support only insert of new tuples and bulk deletes of a large number of old tuples at a time. Inserts are done at the end of the relation representation, that is, new tuples are appended to the relation. Since small deletes and updates do not occur in a data warehouse, large sequences of attribute values can be stored and compressed together as one unit, allowing for better compression than with small sequences.

- **Cost of decompression.** Fetching data from a compressed representation requires *decompression*, which in the simplest compressed representations requires reading all the data from the beginning of a file. Transaction processing queries usually only need to fetch a few records; sequential access is expensive in such a scenario, since many irrelevant records may have to be decompressed to access a few relevant records.

Since data analysis queries tend to access many consecutive records, the time spent on decompression is typically not wasted. However, even data analysis queries do not need to access records that fail selection conditions, and attributes of such records should be skipped to reduce disk I/O.

To allow skipping of attribute values from such records, compressed representations for column stores allow decompression to start at any of a number of points in the file, skipping earlier parts of the file. This could be done by starting compression afresh after every 10,000 values (for example). By keeping track of where in the file the data start for each group of 10,000 values, it is possible to access the i th value by going to the start of the group $\lfloor i/10000 \rfloor$ and starting decompression from there.

ORC and Parquet are columnar file representations used in many big-data processing applications. In ORC, a row-oriented representation is converted to column-oriented representation as follows: A sequence of tuples occupying several hundred megabytes is broken up into a columnar representation called a **stripe**. An ORC file contains several such stripes, with each stripe occupying around 250 megabytes.

Figure 13.15 illustrates some details of the ORC file format. Each stripe has index data followed by row data. The row data area stores a compressed representation of the sequence of value for the first column, followed by the compressed representation of the second column, and so on. The index data region of a stripe stores for each attribute the starting point within the stripe for each group of (say) 10,000 values of

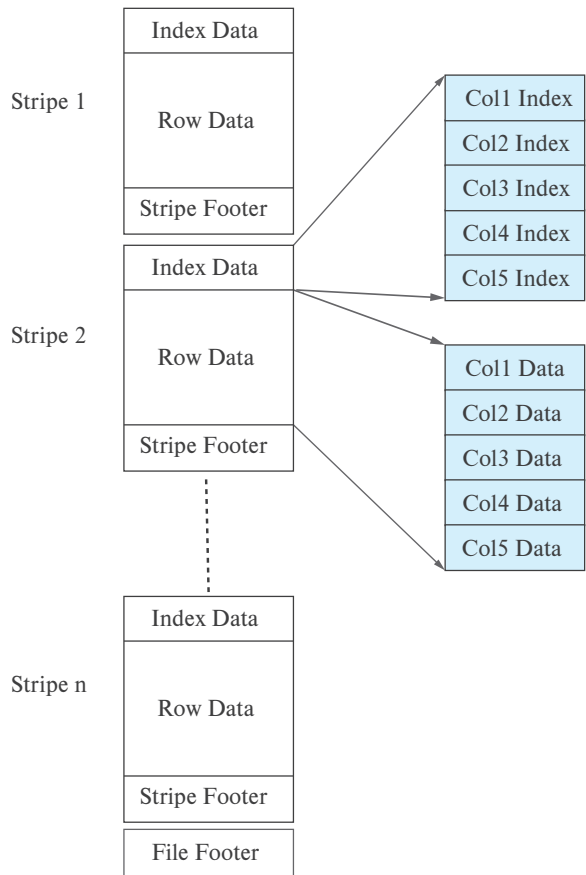


Figure 13.15 Columnar data representation in the ORC file format.

that attribute.⁷ The index is useful for quick access to a desired tuple or sequence of tuples; the index also allows queries containing selections to skip groups of tuples if the query determines that no tuple in those groups satisfies the selections. ORC files store several other pieces of information in the stripe footer and file footer, which we skip here.

Some column-store systems allow groups of columns that are often accessed together to be stored together, instead of breaking up each column into a different file. Such systems thus allow a spectrum of choices that range from pure column-oriented storage, where every column is stored separately, to pure row-oriented storage, where all columns are stored together. The choice of which attributes to store together depends on the query workload.

⁷ORC files have some other information that we ignore here.

requiring more seeks, in contrast to sequential I/O operations with LSM tree variants. For magnetic disk storage, the high cost of seeks results in buffer trees performing worse than LSM trees on write-intensive workloads. LSM trees have thus found greater acceptance for write-intensive workloads with data stored on magnetic disk. However, since random I/O operations are very efficient on SSDs, and buffer trees tend to perform fewer write operations overall compared to LSM trees, buffer trees can provide better write performance on SSDs. Several index structures designed for flash storage make use of the buffer concept introduced by buffer trees.

Another benefit of buffer trees is that the key idea of associating buffers with internal nodes, to reduce the number of writes, can be used with any type of tree-structured index. For example, buffering has been used as a way of supporting bulk loading of spatial indices such as R-trees (which we study in Section 14.10.1), as well as other types of indices, for which sorting and bottom-up construction are not applicable.

Buffer trees have been implemented as part of the **Generalized Search Tree (GiST)** index structure in PostgreSQL. The GiST index allows user-defined code to be executed to implement search, update, and split operations on nodes and has been used to implement R-trees and other spatial index structures.

14.9 Bitmap Indices

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key. We describe key features of bitmap indices in this section but provide further details in Section 24.3.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n , it must be easy to retrieve the record numbered n . This is particularly easy to achieve if records are fixed in size and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Consider a relation with an attribute that can take on only one of a small number (e.g., 2 to 20) of values. For instance, consider a relation *instructor_info*, which has (in addition to an ID attribute) an attribute *gender*, which can take only values **m** (male) or **f** (female). Suppose the relation also has an attribute *income_level*, which stores the income level, where income has been broken up into five levels: *L1*: 0 – 9,999, *L2*: 10,000 – 19,999, *L3*: 20,000 – 39,999, *L4*: 40,000 – 74,999, and *L5*: 75,000 – ∞ . Here, the raw data can take on many values, but a data analyst has split the values into a small number of ranges to simplify analysis of the data. An instance of this relation is shown on the left side of Figure 14.28.

A **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The i th bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A . All other bits of the bitmap are set to 0.

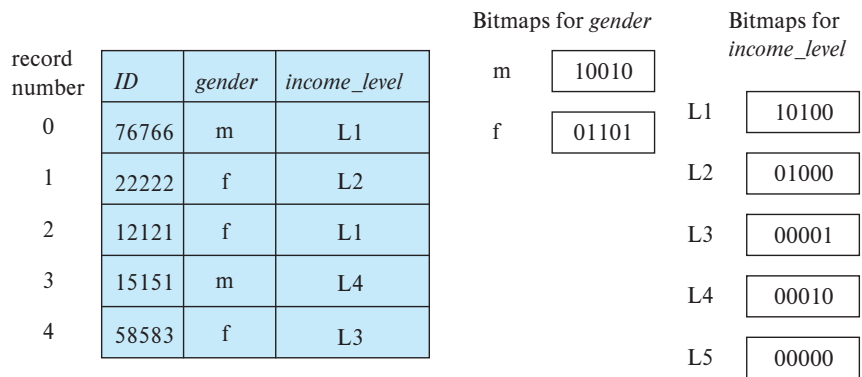


Figure 14.28 Bitmap indices on relation *instructor_info*.

In our example, there is one bitmap for the value **m** and one for **f**. The *i*th bit of the bitmap for **m** is set to 1 if the *gender* value of the record numbered *i* is **m**. All other bits of the bitmap for **m** are set to 0. Similarly, the bitmap for **f** has the value 1 for bits corresponding to records with the value **f** for the *gender* attribute; all other bits have the value 0. Figure 14.28 shows bitmap indices on the *gender* and *income_level* attributes of *instructor_info* relation, for the relation instance shown in the same figure.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value **m** (or value **f**) would be to simply read all records of the relation and select those records with value **m** (or **f**, respectively). The bitmap index doesn't really help to speed up such a selection. While it would allow us to read only those records for a specific gender, it is likely that every disk block for the file would have to be read anyway.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range 10,000 to 19,999. This query can be expressed as

```
select *
from instructor_info
where gender = 'f' and income_level = 'L2';
```

To evaluate this selection, we fetch the bitmaps for *gender* value **f** and the bitmap for *income_level* value **L2**, and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit *i* has value 1 if the *i*th bit of the two bitmaps are both 1, and has a value 0 otherwise. In the example in Figure 14.28, the intersection of the bitmap for *gender* = **f** (01101) and the bitmap for *income_level* = **L2** (01000) gives the bitmap 01000.